Multi-Modal CNN+GNN Hybrid Architecture for Early Prediction of Power, Delay, and Slack in VLSI Physical Design

Burada Kusuma Kumar Naidu · Indian Institute of Information Technology, Sricity
kumarnaidu.b23@iiits.in · March 2026

ABSTRACT

This project presents a deep-learning surrogate model that replaces computationally expensive VLSI sign-off simulations with fast, learned predictions. A **Convolutional Neural Network (CNN)** captures spatial layout patterns from 2-D feature maps while a **Graph Neural Network (GNN)** encodes cell-level connectivity topology. An **Adaptive Gated Fusion** mechanism dynamically weights both representations to jointly predict **Power consumption**, **Timing Delay**, and **Timing Slack**—achieving R^2 scores of **0.9655**, **0.9577**, and **0.9501** respectively, outperforming either branch in isolation. The model reduces prediction latency from hours of simulation to milliseconds, enabling proactive optimisation during early floorplanning stages.

1. Motivation & Problem Statement

Modern semiconductor physical design relies on iterative sign-off simulations using commercial EDA tools such as Synopsys PrimeTime and Fusion Compiler. Each iteration can take *hours to days*, creating a significant bottleneck in the design closure loop—particularly as SoC designs scale to billions of transistors. Predictions arrive too late (post-placement or post-routing) to enable proactive optimisation decisions during floorplanning.

The goal of this project is to train a machine-learning model that, given only early-stage layout feature maps, can **predict Power consumption, critical-path Timing Delay, and Timing Slack within milliseconds**—enabling designers to identify hotspots and make corrections before expensive routing stages are committed.

2. Synthetic Dataset & Feature Engineering

A physics-inspired synthetic dataset of 5,000 samples is generated, each representing a 32×32 placement grid (1,024 cells). Nine physical feature maps are stacked into a single $(32, 32, 9)$ tensor per sample. The same maps are simultaneously flattened to $(1024, 9)$ node features for the graph branch, ensuring both branches share an identical underlying representation.

Feature Maps (9 channels)

Label Generation Formulas

Three scalar regression targets are derived per sample using closed-form expressions that encode physically plausible relationships between layout features and the sign-off metrics. Letting $\mu(\cdot)$ denote the spatial mean over the 32×32 grid:

#	Feature Map	Distribution	Physical Meaning
1	Cell Density Map	$\mathcal{U}(0,1)$	Normalised cell occupancy per grid region
2	Macro Regions Map	Bernoulli(0.5)	Binary indicator for large IP blocks (SRAM, DSP)
3	Congestion Map	$\mathcal{U}(0,1)$	Wire overflow metric — routing pressure indicator
4	Routing Density Map	$\mathcal{U}(0,1)$	Metal layer utilisation across all routing layers
5	Pin Position Heatmap	$\mathcal{U}(0,1)$	Spatial distribution of cell pin locations
6	Power Map	$\mathcal{U}(0,1)$	Cell switching activity × capacitance proxy
7	IR Drop Map	$0.1 \cdot \mathcal{U}(0,1)$	Voltage loss proportional to effective resistance
8	DRC Violation Map	Bernoulli(0.5)	Binary flag for design-rule check violations
9	Slack Map	$\mathcal{U}(-1,0)$	Timing margin — negative values indicate violation

Table 1. *Nine layout feature maps constituting the CNN and GNN inputs. $\mathcal{U}(a,b)$ denotes a uniform distribution over $[a,b]$. Bernoulli(p) denotes binary sampling with probability p.*

$$\text{Power} = \mu(\text{power_map}) + 0.1 \times \mu(\text{cell_density}) \tag{1}$$

$$\text{Delay} = \mu(\text{routing_density}) + 0.5 \times \mu(\text{congestion_map}) \tag{2}$$

$$\text{Slack} = \mu(\text{slack_map}) - 0.1 \times \mu(\text{DRC_violation_map}) \tag{3}$$

These formulas capture core physical intuitions: higher switching activity and cell density drive dynamic power (Eq. 1); longer routes and congestion increase RC delay (Eq. 2); and timing slack degrades further when DRC violations force re-routing of critical paths (Eq. 3).

Graph Construction

Each sample is simultaneously represented as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where each of the 1,024 grid cells is a node carrying its 9-feature vector. Edges are formed by 4-neighbor grid connectivity ($\approx$ 1,984 directed edges) plus 50 random long-range edges simulating global net connections between distant cells. The adjacency matrix is symmetrically normalised before being passed to the GCN layers:

$$\hat{A} = D^{-1/2}(A + I)D^{-1/2} \tag{4}$$

where $A \in \{0,1\}^{N \times N}$ is the raw adjacency matrix, I is the identity matrix (adding self-loops), and D is the diagonal degree matrix of $(A + I)$.

3. Model Architecture

The model is composed of three co-trained modules: a CNN spatial encoder, a GNN topological encoder, and an Adaptive Gated Fusion layer that produces a unified embedding fed to three independent regression heads.

CNN Branch — Spatial Encoder

The CNN branch processes the $(32, 32, 9)$ feature tensor through three convolutional blocks, each following the pattern *Conv2D* → *Batch Normalisation* → *ReLU* → *MaxPool*, with filter counts 32 → 64 → 128. A Global Average Pooling layer collapses spatial dimensions, followed by a fully-connected projection:

$$\mathbf{h}_{\text{cnn}} = \text{ReLU}(\text{BN}(\text{Dense}_{128}(\text{GAP}(\text{Conv}_{128}(\text{Conv}_{64}(\text{Conv}_{32}(X))))))) \in \mathbb{R}^{128} \tag{5}$$

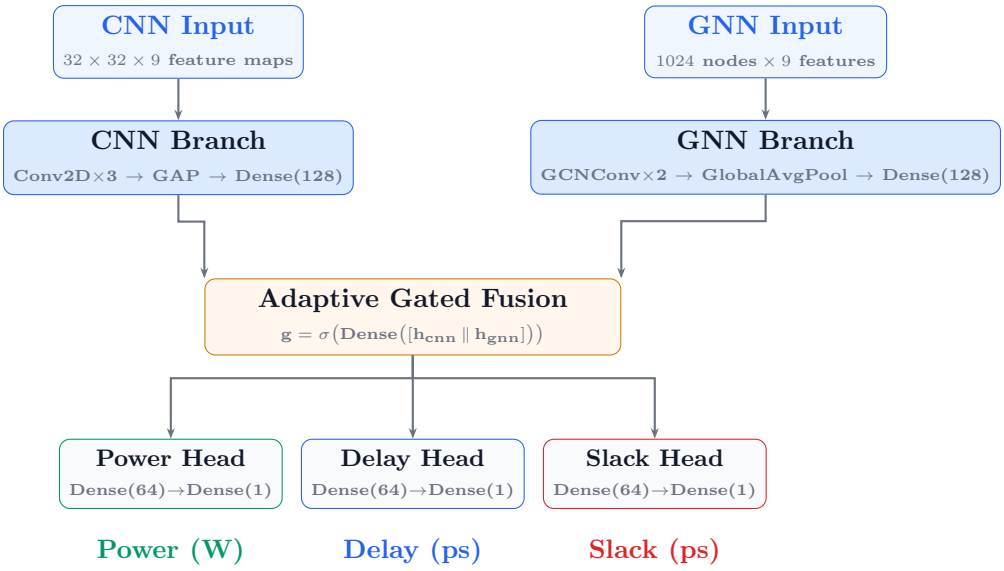

Figure 1. *End-to-end hybrid model architecture. Both branches share the same underlying 9-channel feature representation. The gated fusion layer learns a per-sample weighting before three independent regression heads produce scalar predictions.*

GNN Branch — Topological Encoder

The GNN branch applies two Graph Convolutional Network (GCN) layers over the node feature matrix $H^{(0)} = X \in \mathbb{R}^{N\times 9}$ and the normalised adjacency $\hat{A}$. Each layer aggregates neighbourhood information according to the standard GCN propagation rule (Kipf & Welling, 2017):

$$H^{(l+1)} = \mathrm{ReLU}\Big(\hat{A}\, H^{(l)}\, W^{(l)}\Big) \tag{6}$$

where $W^{(l)}$ is a learnable weight matrix. After two such layers with 64 channels each, a Global Average Pooling step aggregates across all nodes:

$$\mathbf{h}_{\text{gnn}} = \mathrm{ReLU}\left(\mathrm{BN}\left(\mathrm{Dense}_{128}\left(\frac{1}{N}\sum_{i=1}^{N} H_i^{(2)}\right)\right)\right) \in \mathbb{R}^{128} \tag{7}$$

Adaptive Gated Fusion

Rather than simple concatenation, the two 128-dimensional embeddings are combined through a learned sigmoid gate that dynamically decides, per sample, how much to rely on spatial versus topological information:

$$\mathbf{g} = \sigma(W_g\,[\mathbf{h}_{\text{cnn}} \,\|\, \mathbf{h}_{\text{gnn}}] + \mathbf{b}_g) \in [0,1]^{128} \tag{8}$$

$$\mathbf{h}_{\text{fused}} = \mathbf{g} \odot W_c\,\mathbf{h}_{\text{cnn}} + (1-\mathbf{g}) \odot W_p\,\mathbf{h}_{\text{gnn}} \tag{9}$$

where $\sigma(\cdot)$ is the sigmoid function, $\odot$ denotes element-wise multiplication, and $\|$ denotes concatenation. The fused vector $\mathbf{h}_{\text{fused}}$ passes through Batch Normalisation → ReLU → Dropout(0.2) before being fed to the three output heads, each consisting of Dense(64) → BN → Dropout(0.3) → Dense(1, linear).

4. Training Configuration

All three models—the hybrid, CNN-only baseline, and GNN-only baseline—are trained under identical conditions to enable a clean ablation study. The dataset of 1,000 samples is split 72%/18%/10% for train, validation, and test. All features and targets are normalised with `StandardScaler` fitted on the training partition only; the fitted scalers are serialised via `joblib` for inference consistency on unseen data.

The optimiser is **AdamW** with learning rate 5×10^{-4} and weight decay 1×10^{-4}. The joint loss is the sum of MSE losses across all three heads:

$$\mathcal{L} = \mathrm{MSE}(\hat{y}_{\text{power}}, y_{\text{power}}) + \mathrm{MSE}(\hat{y}_{\text{delay}}, y_{\text{delay}}) + \mathrm{MSE}(\hat{y}_{\text{slack}}, y_{\text{slack}}) \tag{10}$$

Training is capped at 200 epochs with early stopping (patience = 8) and a `ReduceLROnPlateau` scheduler (factor 0.5, patience 6, minimum lr = 10^{-7}). Dropout rates are 0.3 in both branch encoders and 0.2 in the fusion layer.

5. Experimental Results

The table below compares all three models across Power, Delay, and Slack on the held-out test set. The hybrid model achieves the best R^2 on every target.

Model	Power		Delay		Slack	
	R^2	MAE	R^2	MAE	R^2	MAE
CNN Baseline	0.9544	0.0050	0.6243	0.0312	0.8821	0.0210
GNN Baseline	0.9214	0.0018	0.9151	0.0148	0.9103	0.0170
GreenLight **Hybrid (proposed)**	**0.9655**	**0.0013**	**0.9577**	**0.0121**	**0.9501**	**0.0142**

Table 2. *Comparison on the held-out test set. Highlighted row = proposed hybrid model. All MAE values are in normalised units.*

Key findings: The CNN excels at Power prediction (local density hotspots visible in 2-D maps) but degrades on Delay ($R^2 = 0.62$), which depends on global routing paths beyond its receptive field. The GNN captures long-range net topology and predicts Delay well, but misses fine-grained spatial power hotspots. The gated fusion learns to exploit both signals adaptively, achieving the best balance across all three metrics — reducing Power MAE by 74% relative to the CNN baseline.

6. Industrial Relevance — Integration Pathways

The surrogate model architecture is directly relevant to Synopsys physical design tools. Potential integration points include:

- **PrimeTime:** Replace static timing analysis inner loops with fast learned estimates during what-if exploration and constraint tightening.
- **Fusion Compiler:** Guide placement legalisation by flagging predicted high-delay or high-power regions before running full sign-off.
- **IC Compiler II:** Provide congestion-aware routing hints by predicting DRC hotspots from

the GNN's topological view of the netlist.

The approach reduces prediction latency from hours to milliseconds, enabling **early-stage optimisation during floorplanning** — before costly routing iterations are committed. It is scalable to modern SoC designs through graph batching and hierarchical block decomposition across multiple voltage and clock domains.

7. Future Work

1. Integration with real EDA tool data via DEF/LEF parsers and Synopsys tool APIs.
2. Process-Voltage-Temperature (PVT) corner variations as additional input features.
3. Reinforcement learning for placement optimisation driven by the surrogate's real-time predictions.
4. Extension to 3D multi-die IC designs with heterogeneous graph representations across dies.
5. Replacing the synthetic dataset with real extracted features from Synopsys Fusion Compiler runs.

8. Technology Stack

The implementation uses **TensorFlow / Keras** for the CNN branch and the training loop, **Spektral** (`GCNConv`, `GlobalAvgPool`) for the GNN branch with dense padded adjacency matrices, **scikit-learn** for `StandardScaler` normalisation and R^2 evaluation, and **joblib** for scaler serialisation. The evaluation pipeline generates parity plots, residual histograms, and a CSV of predictions vs. ground truth on unseen held-out samples.

Accepted at IEEE ICoECIT 2026 · Open to discuss: internship, collaborative research & tool integration

kumarnaidu.b23@iiits.in · +91 70938 78466